



# 类

## ▼ 类的理解

在 C++ 中，类是近似于结构体，但比结构体更高级一种数据类型，它可以包含数据成员和成员函数。

数据成员是类中用于存储数据的成员变量，可以是私有的或公有的。私有成员只能在类的内部访问，公有成员可以在类的内部和外部访问。

成员函数是类中用于操作数据成员的函数，可以是公有的、私有的或受保护的。公有成员函数可以在类的外部直接调用，私有成员函数只能在类的内部调用，受保护的成员函数可以在类的内部和派生类中调用。

类的本质是：一种类型定义，不占据运行时内存空间。

类本身只是一个命名空间，不占据实际内存，只有类的实例在初始化后占据内存，并且类的实例中占据内存的只有成员变量，类的方法与静态成员均由类共享，占据的内存与类的实例无关。

类本身不会生成任何内存内容，其成员函数会被翻译成汇编代码，出现在 `.text` 段。如果没有实例化这个类，且该函数也未被使用，现代编译器甚至可以在优化中将其完全剔除。

- 编译器记录类的：
  - 数据成员布局（偏移）；
  - 方法表（如虚表指针）；
  - 对齐规则；
  - 类型信息；
- 但不会生成任何数据段或堆栈上的变量。

| 对象      | 是否占据运行时空间                 | 是否出现在可执行文件中      | 条件                                 |
|---------|---------------------------|------------------|------------------------------------|
| 类定义     | ✗ 不占空间                    | ✓ 类型信息可能保留（调试信息） | 只要声明了就有，但不一定会进最终 <code>.exe</code> |
| 成员函数    | ✓ 占用 <code>.text</code> 段 | ✓                | 若未内联/未优化掉                          |
| 类的静态变量  | ✓ 如果有定义                   | ✓                | 占用 <code>.data</code> 段            |
| 非静态成员变量 | ✓ 实例化后才占用                 | ✓                | 每个实例各自一份                           |
| 模板类/函数  | ✗ 模板定义不占空间                | ✓ 实例化后编译器才生成代码   | 类似“代码生成器”                          |

通过定义类，可以创建对象，每个对象都是类的一个实例。对象可以访问类中的数据成员和成员函数，可以执行相应的操作。同一个类的不同实例中，所有数据成员都是相互独立的。

对象和实例的定义十分相近，可以理解为：对象为一个抽象的概念，代表了某个类所创建的所有实例，而实例则是某个具体的对象。

## ▼ 单继承

对于普通的成员函数，也可以类似于构造函数，使用“:”来为成员赋值。

“:”用在()于 {} 之间，赋值方式与构造函数相同。

在类的定义时也会使用“:”，此时符号之后表示当前类的父类，可以简单理解为将父类的所有成员与函数都赋值给子类，所有子类拥有所有父类非 `private` 的成员。（子类与父类也叫派生类与基类）

```
class Derived:继承方式 Base{};
```

C++

在派生类构造时，会调用父类的构造函数。

如果基类的默认构造函数已经被覆盖，无法被自动调用，那么在派生类中需要手动实现调用基类构造函数的构造函数。

继承方式遵循“安全性”原则，安全等级只会上升，不会降低，而基类中的 `private` 则不会被继承，所以下列继承方式中都不包括基类的 `private` 成员。

使用 `public` 继承，派生类中的基类成员安全等级不变；

使用 `protected` 继承，派生类中的基类 `public` 成员变为 `protected` 成员，其他不变。

使用 `private` 继承，派生类中的所有非 `private` 成员变为 `private` 成员。即使是 `private` 继承，也不会继承基类的 `private` 成员。

## ▼ 类指针与基类、派生类

虽然基类与派生类本质算是两个类，但是两者的指针可以在一定程度上通用。

派生类的内存空间是一定大于基类的，所以基类指针的步长会小于等于派生类指针。

使用基类指针指向派生类，会发生一定程度的“阉割”，基类指针无法使用派生类中增加的成员或者函数，但能保证指针可以访问的范围小于实际分配的范围，所以不会出错。

如果派生类中重写了基类的虚函数，此时会动态识别指针指向的具体类别并使用派生类中重写的函数，而普通函数则会根据指针类型来执行静态绑定的基类函数。

使用派生类指针指向基类，这种方式极有可能报错，因为指针的可访问范围有可能大于对象的内存大小。如果该派生类没有添加新的成员，那么则不会报错。不过还是应该尽可能避免使用这种方法。

## ▼ 多重继承

```
struct A { int a = 1; };
struct B { int b = 2; };
struct C : A, B {};
```

C++

那么 **C** 会完整地继承 **A** 和 **B** 中的所有非私有成员，包括：

| 类别      | 是否继承                                                                    |
|---------|-------------------------------------------------------------------------|
| 非私有成员变量 | ✓ 是                                                                     |
| 非私有成员函数 | ✓ 是                                                                     |
| 构造函数    | ✗ 不能继承 (C++11 起可以用 <code>using</code> 继承)<br>✓ 所有父类都存在默认构造函数时，会生成默认构造函数 |
| 析构函数    | ✓ 默认生成                                                                  |
| 私有成员    | ✗ 否                                                                     |

## ▼ 构造函数

派生类默认不会自动拥有基类的构造函数

```
struct Base {
    Base(int x) { std::cout << "Base(" << x << ")\n"; }
};

struct Derived : public Base {
    // 什么都没写
};

int main() {
    Derived d(5); // ✗ 错误: Derived 没有合适的构造函数
}
```

C++

但是如果基类的默认构造函数存在且可访问，只要派生类没有自定义任何构造函数，编译器会自动生成一个默认构造函数，并且该默认构造函数会调用所有基类的默认构造函数。

C++

```

struct A {
    A() { std::cout << "A()\n"; }
};
struct B {
    B() { std::cout << "B()\n"; }
};
struct C : public A, public B {
    C() { std::cout << "C()\n"; }
};

int main() {
    C c;
}

```

如果基类的默认构造函数已经被覆盖，那么有两种方式解决：

▼ 显式编写派生类构造函数并手动调用基类构造函数

C++

```

struct A {
    A(int) { std::cout << "A(int)\n"; }
};
struct B {
    B(double) { std::cout << "B(double)\n"; }
};
struct C : public A, public B {
    C() : A(1), B(2.0) { std::cout << "C()\n"; }
};

```

▼ using 显式继承构造函数

C++

```

struct C : public A, public B {
    using A::A;

    // 显式定义构造函数，手动初始化 B
    C(int x) : A(x), B(3.14) { }
};

```

不过 `using Base::Base` 只能继承一个基类的构造函数，不会帮你初始化其他基类或成员变量，所以在多重继承中，依然需要手动调用其他基类的构造函数。

构造顺序遵循：

- 父类构造顺序：按继承声明顺序（不是初始化列表顺序）；
- 成员变量构造：按声明顺序；
- 最后才是派生类本身的构造体；

▼ 命名冲突

当父类的成员出现重名时，需要显示指定命名空间：

C++

```

struct A { int value = 1; };
struct B { int value = 2; };
struct C : A, B { };

int main() {
    C c;
    // std::cout << c.value; // ❌ 错误：二义性
    std::cout << c.A::value << "\n"; // ✅ 显式指定父类
    std::cout << c.B::value << "\n";
}

```

同理，在函数重名/签名冲突时，也需要指定命名空间：

```

struct A { void f(int) { std::cout << "A::f\n"; } };
struct B { void f(double) { std::cout << "B::f\n"; } };
struct C : A, B { };

C c;
c.f(1); // ❌ 错误：二义性，不知道是 A::f 还是 B::f
c.A::f(1); // ✅
c.B::f(1.5); // ✅

```

## ▼ 菱形继承

如果多重继承的父类之间存在共同的父类，那么就会出现菱形继承：

```

class A { public: int x; };
class B : public A {};
class C : public A {};
class D : public B, public C {}; // ⚠️ D 拥有两个 A 的副本

D d;
d.x = 5; // ❌ 二义性，x 存在两份

```

这种情况下，派生类的基类都会尝试构造它们的基类，从而出现冲突。

如果可能出现这种情况，需要使用虚继承：

```

class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {}; // ✅ 只有一份 A

```

## ▼ 虚继承

在继承体系中，最底层的那个派生类（也叫**最终派生类**，最外层子类）被称为**最派生类**（most derived class），它是**负责初始化所有虚基类的唯一类**。

虚继承中“虚基类”必须由“最派生类”负责构造。

如果虚基类没有默认构造函数，最派生类必须显式调用其构造函数，否则会编译失败

C++

```

struct V {
    V(int x) { std::cout << "V(" << x << ")\n"; }
};

struct A : virtual public V {
    A() { std::cout << "A()\n"; }
};

struct B : virtual public V {
    B() { std::cout << "B()\n"; }
};

struct C : public A, public B {
    // 需要手动调用 V 的构造函数！
    C() : V(42), A(), B() {
        std::cout << "C()\n";
    }
};

```

虚继承成功避免了最派生类的基类构造它们的基类时出现冲突的情况。

虚继承中，虚基类的构造交由最派生类进行，如果中间类的构造函数中手动调用了虚基类的构造函数，编译器会无视这个构造，但是不会报错。

## ▼ 参数包继承

在可变参数模板中，存在包含多种类型的类型参数包，该参数包也可以被继承，并会被认为是多重继承。

```

// 重载结构体定义
template<class... Ts>
struct Overloaded : Ts... {
    using Ts::operator()...;
};

```

C++

## ▼ 成员函数调用

成员函数一般指类中的非静态函数。

在编译层面上看，类成员函数并不属于这个类，其本质上就是个命名空间中的普通函数（虚函数除外），只是在符号表中带有类空间信息的函数。

在编译阶段，会根据调用处的对象类型，确定调用哪个类的成员函数以及该函数符号，生成的汇编代码中的调用指令会调用上述唯一的符号名，并在链接时通过符号名找到对应函数的实现地址。

当某个实例调用成员函数时，成员函数中的 `this` 指针就会指向这个实例的地址。所以如果类内的成员函数中有成员与函数参数重名，可以使用 `this` 指针来访问实例内的成员变量。

对于普通的 `public` 成员函数来说，必须通过实例来调用，即使这个函数并没有用到这个实例中的任何成员。

之所以必须通过实例来调用普通的成员函数，这涉及到了非静态成员函数的工作原理：

C++

```
class Test{
public:
    void out()&{
        cout<<"hello world!";
    }
};
Test test;
test.out(); //可以理解为Test::out(&test)
```

编译器会把 `test.out()` 转换为类似于 `Test::out(&test)` 这样的形式，其中 `&test` 作为隐式的 `this` 指针传递。

因为普通成员函数需要一个所属类的实例作为第一个参数，所以其调用一般无法脱离实例。

在 C++23 开始，可以使用 `this` 显式对象声明，在声明普通成员函数时就显式声明 `this` 指针，`void out(this const Test&)`。

对于 `private` 的成员，则无法直接通过实例调用。

在类中，一般需要保留一个 `public` 成员函数，作为访问的入口，而通过这个 `public` 函数则可以调用同一个类内的 `private` 函数或者修改 `private` 的变量。

```
class Test{
public:
    void public_out(){
        private_out();
    }
private:
    void private_out(){
        cout<<"hello world!";
    }
};
//如果想要调用out()函数，必须通过一个实例来调用，所以需要先实例化一个对象
/*
Test my_test;
my_test.private_out(); 错误，无法调用。
my_test.public_out(); 正确，通过public函数调用private函数。
*/
```

C++

## ▼ 常量成员函数

在成员函数的定义中经常会使用 `const` 修饰，使用 `const` 修饰的成员函数不能更改任何成员。

```
class Test{
private:
    int data;

public:
    Test(int a):data(a){}

    int GetData()
```

C++

```

{
    data++;
    return data;
}

int GetData() const
{
    return data;
}

//const int GetData(){}表示返回值为const int类型的函数。
};

Test t1(1);
const Test t2(1);
int a=t1.GetData(); //a=2
int b=t2.GetData(); //b=1

```

const 在使用时需要在 `()` 传参之后，是因为 const 是修饰 this 指针的，表示不能修改实例中的任何成员。

const 类型的实例无法调用非 const 的成员函数，不过非 const 的实例可以调用 const 成员函数，所以无论实例是否为 const 类型，都可以调用 const 函数，所以不涉及到修改成员的函数都尽可能使用 const。

不过为了避免 const 实例调用非 const 类型的成员函数从而报错，可以对函数进行重载，即同一个类中可以存在除了“是否使用 const 修饰”以外完全相同的两个函数。在调用该函数时会根据实例是否为 const 类型来调用对应的函数。

## ▼ 静态成员

在类中使用 static 修饰的变量为静态变量，static 修饰的函数为静态函数。

静态成员并不属于某一个实例，而是所有实例共享。

想要访问静态成员，可以直接通过类的作用域来访问，也可以通过实例来访问。但即使通过实例访问，静态变量也依然不属于这个实例。

```

class Test{
public:
    static void out(){
        cout<<"hello world!";
    }
};
/*
Test::out();
*/

```

C++

需要注意的是，在类的空间内并不能对成员变量进行定义，所以类内的成员变量本质都是声明，它们会在实例化时再被创建。

但是静态成员变量并不属于某个实例，所以我们需要单独对其进行定义。



定义时不能在类内，也不能在与类平级的其他作用域内，必须在更上级作用域内定义。一般情况下可以直接在全局作用域内定义。

```
class Test{
public:
    static int a;
};
int Test::a=0;
```

C++

## ▼ 成员函数引用限定

重载参数类型相同的两个成员函数需要它们同时具有或缺少引用限定符，即如果成员函数使用了 `&` 或 `&&`，则不能再有不使用引用限定符的重载。

```
class Test{
public:
    void foo(int);
    void foo(int) &&;
    // 错误，重载参数类型相同的两个成员函数需要它们同时具有或缺少引用限定符
};
```

C++

因为不加引用限定符的成员函数可以接受任何类型，所以存在该函数时，编译器无法判断具体使用哪个函数。

如果需要对左值和右值分开处理，则需要对 `&` 与 `&&` 两种情况进行重载。

```
class Test{
public:
    //这个声明中的 & 表示左值限定成员函数。该成员函数只能通过左值对象调用，不能通过右值对象调用。
    void foo(int) &;
    //这个声明中的 && 表示右值限定成员函数。该成员函数只能通过右值对象调用，不能通过左值对象调用。
    void foo(int) &&;
};
```

C++

### ▼ 左值限定

```
class A {
public:
    void foo(int) const & {
        std::cout << "Called on an lvalue" << std::endl;
    }
};

A a;
a.foo(10);    // OK, a 是左值
A().foo(10);  // 错误, A() 是右值, 不能调用左值限定的成员函数
```

C++

### ▼ 右值限定

C++

```
class A {
public:
    void foo(int)&& {
        std::cout << "Called on an rvalue" << std::endl;
    }
};

A().foo(10); // OK, A() 是右值, 可以调用右值限定的成员函数
A a;
a.foo(10);   // 错误, a 是左值, 不能调用右值限定的成员函数
```

二者可以在同一个类中声明, 构成函数重载。

不过 `void foo(int) &;` 的适用范围太窄, 既不能被右值对象调用, 也不能被 `const` 对象调用, 所以一般更倾向于使用 `void foo(int) const &;`。

`void foo(int) const &;` 既包含 `void foo(int) &;` 可以接受的范围, 也可以在有 `void foo(int) &&;` 重载的情况下接受右值。

```
class Test{
public:
    void foo(int) const &;
    void foo(int) &&;
};
```

C++

## ▼ 成员指针

普通指针 (如 `int*`) 存储的是 CPU 内存地址, 是一个绝对数值。

而成员指针存储的是偏移量, 它描述的是某个成员相对于类起始地址的“相对位置”, 只有提供一个具体的对象实例时, 成员指针才能计算出真正的地址。

具体语法为: 在指针类型的数值类型与\*之间加上类的命名空间, `T C::*`, 表示 `C` 类中一个 `T` 类型的指针。如 `int MyClass::*` 表示 `MyClass` 类中一个 `int` 类型的指针。

由于成员在类中的布局是不连续的 (可能有对齐填充), 所以不能对成员指针进行算术运算, 否则可能会指向未知的区域。

尽管成员指针不是地址, 但它依然需要表示一个特殊状态来表示其指向的地址是否有效, 为了方便程序员判断一个成员指针是否已被初始化, 编译器允许将其隐式转换为布尔值。

在 C++11 之前, 成员指针的转换规则非常“死板”, 只有以下几条路径:

- 转 `bool`: 这是标准特许的。成员指针可以直接用于 `if`、`while`。
- 转同类的其他成员指针: 支持逆变/协变转换 (基类与派生类成员指针的转换)。
- 转自己: 赋值给同类型的成员指针变量。
- 禁止转 `int`: 标准没有定义从 `T C::*` 到任何整型 (`int`, `long`, `char`) 的隐式转换。

在 C++11 之前的 `nullptr` 中, 曾利用这个特性实现 Safe Bool, 详情见 [【进阶语法→nullptr】](#)。

